

Name	Nitish Kumar
Project Title	Employee Management System
Technology Used	Python (file-based storage)
Report Date	July 26, 2026

Employee Management System

A Command-Line Application in Python using Text File Storage

File analyzed: Employee management system.py

1. Introduction

The Employee Management System is a menu-driven command-line application written in Python. It utilizes an Object-Oriented approach by grouping functionalities within an `Employee` class using static methods. The application allows a user to maintain a database of employee records—adding, viewing, searching, updating, and deleting entries—along with a feature to calculate the total salary payout. All data is persisted to a standard text file (`project.txt`), demonstrating core programming concepts including file handling, string manipulation, control flow, and basic CRUD (Create, Read, Update, Delete) operations.

2. Objectives

- To design a lightweight system for storing and managing employee records using basic file handling.
- To implement standard CRUD operations by reading, writing, and parsing text files.
- To utilize Python classes and static methods for organized code structure.
- To provide a simple, interactive text-based menu for ease of use.
- To implement basic data aggregation (calculating total salary).

3. Tools and Technologies Used

- **Language:** Python 3
- **Libraries:** Built-in Python functions (no external modules required)

- **Storage:** Text file (`project.txt`)
- **Interface:** Command-line (text-based menu)

4. System Overview

The program stores each employee record as a comma-separated string on a new line in `project.txt` , containing four fields: Employee ID, Name, Age, and Salary. The application runs a menu-driven loop that displays a 6-option menu. Based on the user's input, the script dispatches the request to the corresponding static method within the `Employee` class.

5. Module (Function) Descriptions

Function	Purpose
<code>Employee.work()</code>	Prints the interactive main menu options to the console.
<code>Employee.add_employee()</code>	Prompts the user for employee ID, name, age, and salary, then appends this data as a formatted string to <code>project.txt</code> .
<code>Employee.display_employee()</code>	Reads all lines from the text file, splits the comma-separated values, and prints them in an aligned, formatted table.
<code>Employee.search_employee()</code>	Searches for an employee by ID and prints their specific details if a match is found in the text file.
<code>Employee.update_employee()</code>	Locates an employee by ID, prompts for a new Name and Age, and rewrites the entire file with the updated record while keeping others intact.
<code>Employee.delete_employee()</code>	Removes an employee's row by ID by filtering it out and rewriting the text file with the remaining records.
<code>Employee.calculate_salary()</code>	Iterates through all saved records, parses the salary field as an integer, and calculates the total salary of all employees.

6. Data Structure

Each record in `project.txt` consists of four data points separated by commas:

`Emp_id` , `Name` , `Age` , `Salary`

Example of stored data:

```
101, John Doe, 28, 55000
102, Jane Smith, 32, 62000
```

7. Program Flow

1. The `while True:` loop begins by calling `Employee.work()` to display the 6-option menu.
2. User input is read as an integer to determine the desired operation.
3. The input is routed to the corresponding static method via `if-elif-else` branching.
4. Each method opens `project.txt` in either append (`"a"`), read (`"r"`), or write (`"w"`) mode.
5. Strings are parsed using the `.strip().split(",")` string methods to isolate individual data variables.

8. Sample Code Snippet

The search function illustrates the string parsing and record-lookup pattern used throughout the program:

```
@staticmethod
def search_employee():
    ID=input("Enter your Emp_id: ")
    found=False
    with open("project.txt","r") as file:
        for line in file:
            Emp_id, Name, Age, Salary = line.strip().split(",")
            if ID == Emp_id:
                print("====This is your details====")
                print("\n{:<8}{:<15}{:<8}{:<10}".format("ID", "NAME", "AGE",
"SALARY"))
                print("-" * 40)
                print("{:<8}{:<15}{:<8}{:<10}".format(Emp_id, Name, Age, Salary))
                found=True
                break
    if not found:
        print("There are not exist in this list")
```

9. Strengths

- **Object-Oriented Structure:** Uses a class (`Employee`) and `@staticmethod` decorators to logically group related functions.
- **Formatted Output:** Utilizes Python's string `.format()` method to create clean, aligned console tables for viewing records.
- **Data Aggregation:** Includes a mathematical function (`calculate_salary`) to derive new insights (total payroll) from stored data.
- **Simplicity:** Relies purely on standard file I/O operations without needing the `csv` module, demonstrating a strong understanding of string manipulation.

10. Limitations

- **No File Check on Startup:** If `project.txt` does not exist prior to running a read operation, the program will crash with a `FileNotFoundError`.
- **Data Validation:** No error handling for invalid input (e.g., entering letters for Age or Salary).
- **Loop Execution:** The current `while True:` block ends with an unconditional `break` statement outside the `if-elif` chain, meaning the program terminates after a single operation.
- **Inefficient Updates:** Update and Delete operations require rewriting the entire file, which scales poorly with large datasets.

11. Future Enhancements

- Implement `try-except` blocks to handle file missing errors and invalid user inputs (like `ValueError` during integer conversion).
- Fix the main loop so the program continues to prompt the user for actions until an explicit "Exit" option is chosen.
- Transition from manual string splitting to using Python's built-in `csv` module for more robust data handling.
- Add a feature to check for duplicate `Emp_id` entries before adding a new employee.

12. Conclusion

The Employee Management System successfully demonstrates practical Python programming concepts, including file handling, string parsing, class organization, and control flow. By implementing a full suite of

CRUD operations alongside a salary calculation feature, it serves as a solid, functional command-line application that establishes a strong foundation for more complex database management projects.

Submitted by:

Nitish Kumar

Submitted to:

Upskillcampus

GitHub Repository:

<https://github.com/Nitish-Bytes/Upskillcampus/blob/main/Employee%20management%20system.py>